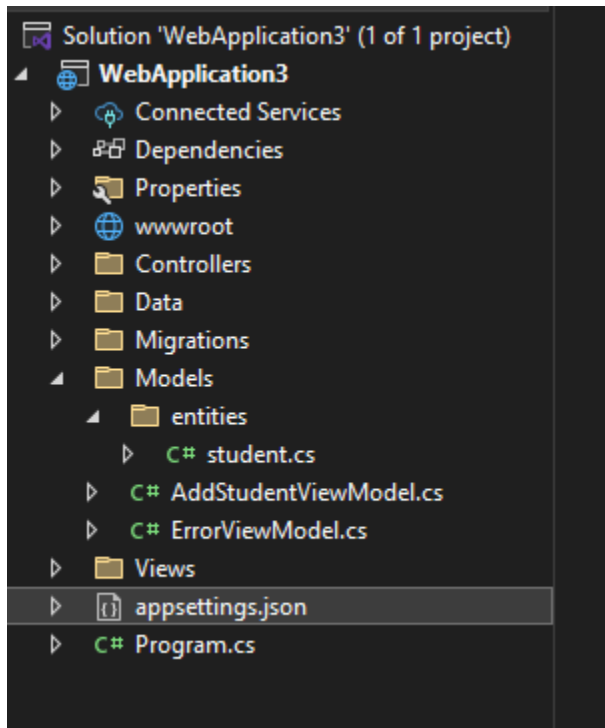


CRUD - ASP.NET MVC

Step-by-step guide:

1. Create an MVC project as given in your lab manual
2. In the Models folder, create a folder named entities, and inside that, create a class named student.cs containing the schema of the table,
Later, create a class in Models folderer with the AddStudentViewModel.cs class, which acts as a viewModel



And add the following code in the student.cs
namespace WebApplication3.Models.entities

```
{
    public class student
    {
        //property
        public Guid Id { get; set; } //C# core struct used to create primary keys
        public string Name { get; set; }
        public string Email { get; set; }
        public string Phone { get; set; }
        public bool Subscribed { get; set; }
    }
}
```

In AddStudentViewModel, add this:

```
namespace WebApplication3.Models
```

```
{  
    public class AddStudentViewModel  
    {  
        public string Name { get; set; }  
        public string Email { get; set; }  
        public string Phone { get; set; }  
        public bool Subscribed { get; set; }  
    }  
}
```

3. Inside the data folder, create a DbContext file. Know the prerequisites (Entity framework dependencies installation is required to use these features). Create a file named ApplicationDbContext inheriting from DbContext using Microsoft.EntityFrameworkCore;

```
using WebApplication3.Models.entities;
```

```
namespace WebApplication3.Data
```

```
{  
    public class ApplicationDbContext: DbContext  
    {  
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options):  
base(options)  
        {  
  
        }  
        //referencing the student entity  
        public DbSet<student> students { get; set; }  
    }  
}
```

Establish Database Connection and verifying it , Go to Program.cs file and edit it

```
using Microsoft.EntityFrameworkCore;
using WebApplication3.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

//injecting Dbcontext into the program.cs telling which db to use
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("StudentPortal")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Add the following code to the correct line matching the image
//injecting Dbcontext into the program.cs telling which db to use
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("StudentPortal")));

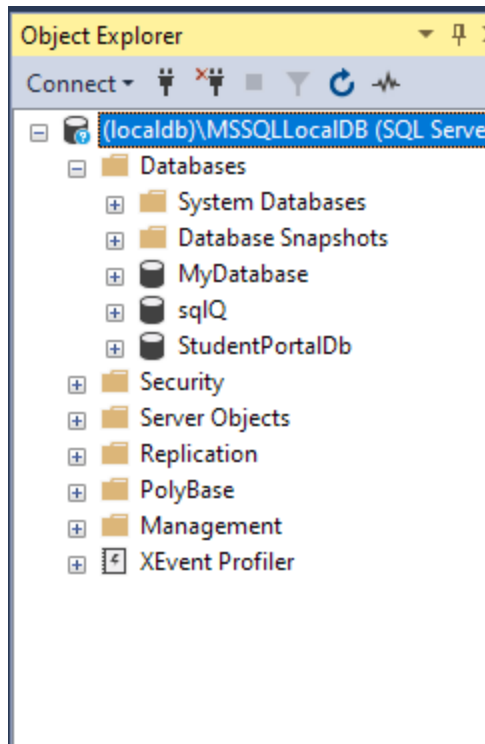
5. Now In AppSettings.json, create a connection string and database connectivity

```
https://json.schemastore.org/appsettings.json
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "StudentPortal": "Server=(localdb)\\MSSQLLocalDB; Database=StudentPortalDb; Trusted_Connection=True; TrustServerCertificate=True "
  }
}
```

```
"ConnectionStrings": {
  "StudentPortal": " Server=(localdb)\\MSSQLLocalDB; Database=StudentPortalDb;
Trusted_Connection=True; TrustServerCertificate=True "
}
```

6. Run Migrations To reflect changes in database , In package manager wire

Add-Migration MigrationName run it then Update-Database and run this will create database in sql server management studio



7. To perform CRUD operation, create controllers and views, and then run it on the browser to see the proper working

Below is the Student Controller:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using WebApplication3.Data;
using WebApplication3.Models;
using WebApplication3.Models.entities;

namespace WebApplication3.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext dbContext;

        public StudentController(ApplicationDbContext dbContext)
        {
            this.dbContext = dbContext;
        }

        [HttpGet]
```

```

public IActionResult Add()
{
    return View(); // will look for Views/Student/Add.cshtml
}

[HttpPost]
public async Task<IActionResult> Add(AddStudentViewModel viewModel)
{
    var student = new student //coming from the entities folder
    {
        Name = viewModel.Name,
        Email = viewModel.Email,
        Phone = viewModel.Phone,
        Subscribed = viewModel.Subscribed
    };

    //dbContext.students is coming from student table defined in ApplicationDbContext
    await dbContext.students.AddAsync(student);
    //save student to the database
    await dbContext.SaveChangesAsync();

    return View();
}

//retrieving list of records from db

[HttpGet]
public async Task<IActionResult> List()
{
    var students = await dbContext.students.ToListAsync();
    return View(students);
}

[HttpGet]
//will have to pass guid to get the information
public async Task<IActionResult> Edit(Guid id)

{
    //using db context to retrieve
    //findAsync will find the results with matching id
    var students = await dbContext.students.FindAsync(id);
    return View(students);
}

```

```

[HttpPost]
public async Task<IActionResult> Edit(student viewModel)

{
    //using db context to retrieve
    //findAsync will find the results with matching id
    var students = await dbContext.students.FindAsync(viewModel.Id);
    if(students is not null)
    {
        students.Name = viewModel.Name;
        students.Email = viewModel.Email;
        students.Phone = viewModel.Phone;
        students.Subscribed = viewModel.Subscribed;

        await dbContext.SaveChangesAsync();
    }
    return RedirectToAction("List", "Student");
}

[HttpPost]
public async Task<IActionResult> Delete(student viewModel)
{
    var students = await dbContext.students.FindAsync(viewModel.Id);
    if (students is not null)
    {
        dbContext.students.Remove(students);
        await dbContext.SaveChangesAsync();
    }

    return RedirectToAction("List", "Student");
}
}
}

```

Razor Views of corresponding controllers:

Add.cshtml

@*

For more information on enabling MVC for empty projects, visit
<https://go.microsoft.com/fwlink/?LinkID=397860>

*@

@{
}

@model WebApplication3.Models.AddStudentViewModel

<h1>Add Student</h1>

<form method="post">

<div class="mt-3">

<label class="form-label">Name</label>

<input type="text" class="form-control" asp-for="Name" />

</div>

<div class="mt-3">

<label class="form-label">Email</label>

<input type="email" class="form-control" asp-for="Email" />

</div>

<div class="mt-3">

<label class="form-label">Phone</label>

<input type="text" class="form-control" asp-for="Phone" />

</div>

<div class="mt-3">

<input type="checkbox" class="form-check-input" id="Subscribed" />

<label class="form-check-label" for="Subscribed" asp-for="Subscribed">Subscribed</label>

</div>

<div class="mt-3">

<button type="submit" class="btn btn-primary">Save</button>

</div>

</form>

Edit.cshtml

@*

For more information on enabling MVC for empty projects, visit
<https://go.microsoft.com/fwlink/?LinkID=397860>

*@

@{
}

@if (Model is null)

{

```
    <p> No students found with the given id</p>
  }
  else
  {
```

```
@model WebApplication3.Models.entities.student
```

```
    <h1>Edit Student</h1>
    <form method="post">

        <div class="mt-3">
            <label class="form-label">Id</label>
            <input type="text" class="form-control" asp-for="Id" />
        </div>
        <div class="mt-3">
            <label class="form-label">Name</label>
            <input type="text" class="form-control" asp-for="Name" />
        </div>

        <div class="mt-3">
            <label class="form-label">Email</label>
            <input type="email" class="form-control" asp-for="Email" />
        </div>

        <div class="mt-3">
            <label class="form-label">Phone</label>
            <input type="text" class="form-control" asp-for="Phone" />
        </div>

        <div class="mt-3">
            <input type="checkbox" class="form-check-input" id="Subscribed" />
            <label class="form-check-label" for="Subscribed" asp-for="Subscribed">Subscribed</label>
        </div>
        <div class="mt-3">
            <button type="submit" class="btn btn-primary">Save</button>
            <button type="submit" class="btn btn-danger ms-3"
                asp-action="Delete"
                asp-controller="Student"
                >Delete</button>
        </div>
    </form>
}
```


List.cshtml

@*

For more information on enabling MVC for empty projects, visit
<https://go.microsoft.com/fwlink/?LinkID=397860>

*@

@{

}

<!--referencing the student model to be in the list form-->

@model List<WebApplication3.Models.entities.student>

<h1>Students</h1>

<table class="table">

<thead>

<tr>

<th>Id</th>

<th>Name</th>

<th>Email</th>

<th> </th>

</tr>

</thead>

<tbody>

@foreach (var student in Model)

{

<tr>

<td>@student.Id</td>

<td>@student.Name</td>

<td>@student.Email</td>

<td>@student.Subscribed</td>

<td>

<a asp-controller="Student" asp-action="Edit" asp-route-id = "@student.Id">Edit

</td>

</tr>

}

</tbody>

</table>